

COMP47910

MFA Assignment

Implement Multi-Factor Authentication (MFA) into the Spring-Boot project [Registration Login Springboot Security Thymeleaf](#) created by [RameshMF](#).

Student Details

- Name: Conor Dunne
- Number: 17379526
- Email: Conor.Dunne7@ucdconnect.ie

Additional Imports

Group	Artifact	Version	Purpose	MVN Repository
com.warrenstrange	GoogleAuth	1.5.0	implements the Time-based One-time Password (TOTP) algorithm	Link
com.google.zxing	JavaSE	3.5.3	Generate a Base64 Encoded format of a QR Code image	Link

Running the Application

Setup MySQL Prior to running

If you already have a MySQL instance running, Update the details in `application.properties`. If not, follow the instructions below:

1. Log into your local MySQL Service
2. Create a database `login_system`
3. Create a user `comp47910` with the password `Mysql@123` in your MySQL instance.
4. Grant user permission to access the database.

Commands Required:

```
mysql> CREATE DATABASE login_system;
mysql> CREATE USER 'comp47910'@'localhost' IDENTIFIED BY 'Mysql@123';
mysql> GRANT ALL PRIVILEGES ON login_system.* TO 'comp47910'@'localhost';
mysql> FLUSH PRIVILEGES;
```

Starting the Application

If you are unable to execute the application using your IDE, navigate to the root directory of the application and start it using Maven.

Commands Required:

```
$ mvn spring-boot:run
```

MFA Implementation

When a new user registers, a random secret is generated for them. They are then redirected to `mfa.html` where a QR code is displayed. By scanning this code using a mobile authentication app, MFA is setup for the user. When the user logs in, this MFA code is required to login. It is not possible to opt out of this.

New Files

- `mfa.html`: was added to display a QR code during MFA setup.
- `GAService.java`: A new service to generate and verify an authentication code using a `GoogleAuthenticator`, as well as generating a Base64 encoded image of a QR code. Created by Shishir Karki in their Medium blog.
- `CustomWebAuthenticationDetailsSource.java`: captures the HTTP request during authorization and returns the MFA code submitted by the user.
- `CustomWebAuthenticationDetails.java`: Default web authentication details component for `CustomWebAuthenticationDetailsSource.java`.
- `CustomAuthenticationProvider.java`: Custom `DaoAuthenticationProvider` to authenticate user credentials. Required to add MFA authentication.

Updated Files

- `login.html`: A form input 'MFA Code' identified by the field name `code` was added.
- `User.java`: A new field `secret` to store the users MFA secret was added.
- `UserDto.java`: A new field `secret` to transfer the users MFA secret between the client and server was added.
- `AuthController.java`: The function `registration` was updated to retrieve the user's secret on creation and send to `mfa.html` in Base64 encoded format.
- `SpringSecurity.java`: New `authManager` function initiates the `CustomAuthenticationProvider` which is implemented using a new `authManager` function. The function `configureGlobal` was no longer required and was removed.

References

- Baeldung Post: [Two Factor Auth with Spring Security](#)
- Medium Blog: [How To Implement Multi-Factor Authentication with Spring Security](#) by Alexander Obregon
- Medium Blog: [Implementing 2Factor TOTP Using Google Auth in Spring Boot](#) by Shishir Karki